

Next.js Take-Home Assessment

Multi-Tenant Ops Platform

Time Expectation

3 hours

Difficulty

Intermediate

1. Technology & Hard Constraints

- Next.js (App Router) with TypeScript
 - Server Components by default; Client Components only when required
 - PostgreSQL with Prisma ORM
 - Authentication using Auth.js (NextAuth) or secure cookie-based sessions
 - Docker Compose must start app + database
 - Server Actions preferred for mutations
 - Automated testing is mandatory (unit, integration, e2e)
-

2. Multi-Tenant Architecture (Critical)

The application must support multiple tenants (organizations). Tenant isolation is mandatory. Any cross-tenant data leakage is a failure.

Data Models

- Tenant: id, name, slug
- User: id, email, name
- Membership: userId, tenantId, role (admin | engineer | viewer)

Rules

- A user may belong to multiple tenants

- Tenant context is selected via `/t/[tenantSlug]/...`
 - Every read and write must be tenant-scoped
 - Authorization checks must run server-side
 - Tenant enforcement must occur at the data-access layer (Prisma middleware or repository layer)
-

3. Incident Management

Incident Fields

- id
- title
- severity (SEV1–SEV4)
- status (open | mitigated | resolved)
- service
- environment (dev | staging | prod)
- createdAt, updatedAt
- createdBy, assignee
- tags

Incident List

- URL-driven filters: status, severity, environment, service, tag, assignee
- Full-text search on title and service
- Cursor-based pagination
- Bulk actions: assign engineer, change status
- Saved Views per user per tenant

Incident Details

- Timeline events: note, action, status_change
 - Enforced status transitions:
 - open → mitigated → resolved
 - open → resolved allowed
 - resolved → open not allowed
 - Status changes must be transactional and auto-insert a status_change event
 - File attachments with stored metadata
-

4. Feature Flags

Feature Flag Model

- Scoped per tenant and per environment
- Rules stored as JSONB and validated with Zod
- Supported rules:
 - Percent rollout
 - Allowlist
 - Logical composition (AND/OR)

Evaluation Engine

- Deterministic evaluation using stable hashing (userId + flagKey)
 - Input: userId, environment, service
 - Output: enabled/disabled with explanation trace
 - UI tool for manual evaluation is required
-

5. Realtime & Background Processing

Realtime

- Incident detail pages must update without refresh
- Implement using Server-Sent Events or WebSockets
- Streams must be authenticated and tenant-scoped

Background Jobs

- Database-backed job queue
 - Worker process polls and processes jobs
 - Simulate attachment antivirus scanning and notifications
-

6. Security, Reliability & Observability

- Audit log every mutation (actor, tenant, before/after, timestamp)
 - CSRF protection for session-based authentication
 - Rate limiting for write operations
 - Prevent XSS in user-generated content
 - Avoid N+1 queries and ensure efficient Prisma usage
-

7. Testing Requirements

- Unit tests: business logic (rules engine, hashing, status transitions)
 - Integration tests: tenant isolation, transactions, audit logging
 - End-to-end tests: full flows including realtime behavior
-

8. Seed Data & Setup

- Seed at least 2 tenants and 3 users with cross-tenant membership
 - Create 40+ incidents and 10+ feature flags per tenant
 - Provide `docker-compose.yml` and `.env.example`
-

9. Submission

- GitHub repository link
- README must document:
 - architecture and tenant enforcement
 - caching and invalidation strategy
 - realtime design
 - security decisions
 - tradeoffs and next steps
- Partial completion is acceptable if clearly documented